ITC Generator — Consolidated Power Projects

A web-based tool for generating pre-filled Inspection and Test Certificate (ITC) Excel files for high voltage commissioning work. Built and maintained by CPP's commissioning team.

What Is This?

The ITC Generator is an internal web application that replaces the manual process of copying and filling out Excel ITC templates. A commissioning officer selects a client, enters project details, chooses equipment types and panel numbers (or uploads an index spreadsheet), and the app generates individual pre-filled Excel ITC files ready to use on site.

Live URLs

Service	URL
Frontend (web app)	Your Vercel URL
Backend (API)	https://itc-generator.onrender.com
GitHub Repository	https://github.com/chrisheron086-svg/itc-generator

---

Tech Stack

Layer	Technology
Frontend	React 18, styled-components, SheetJS (xlsx)
Backend	Python 3.11, FastAPI, openpyxl
Hosting (Frontend)	Vercel (free tier)
Hosting (Backend)	Render (free tier — sleeps after 15 min inactivity)
Version Control	GitHub
Excel Generation	openpyxl

Layer	Technology
Fonts & Branding	Montserrat (CPP brand font), CPP Red `#9E053B`, CPP Gold `#DFB200`

Project Structure

```
itc-generator/
├── backend/
|   ├── main.py                          # FastAPI app — all API endpoints
|   ├── requirements.txt                 # Python dependencies
|   ├── Dockerfile                       # Docker config for backend
|   ├── generators/
|   |   ├── __init__.py
|   |   ├── base.py                      # Shared copy/fill logic for all
templates
|   |   ├── circuit_breaker.py           # Circuit Breaker specific generator
|   |   └── generic.py                   # Generator for all other equipment types
|   └── templates/
|       ├── Primary_ITCs/                # Default primary equipment templates
|       |   ├── Circuit_Breaker.xlsx
|       |   ├── Current_Transformer.xlsx
|       |   ├── Voltage_Transformer.xlsx
|       |   ├── Power_Transformer.xlsx
|       |   ├── Isolator.xlsx
|       |   ├── Surge_Arrestor.xlsx
|       |   ├── Neutral_CT.xlsx
|       |   ├── Earth_Switch.xlsx
|       |   ├── OWS.xlsx
|       |   ├── NET.xlsx
|       |   ├── Aux_TF.xlsx
|       |   ├── BESS_PCS.xlsx
|       |   ├── AC_Board.xlsx
|       |   ├── DC_Panel.xlsx
|       |   └── Feeder_Panel.xlsx
|       ├── Protection_Secondary_ITCs/   # Secondary/protection equipment
templates
|       |   └── SEL_751_Feeder_Relay.xlsx
|       └── [Client_Folder_Name]/        # Client-specific templates (one folder
per client)
|           ├── Circuit_Breaker.xlsx
|           └── ...
```

```
├── frontend/
|   ├── package.json
|   ├── public/
|   |   └── index.html
|   └── src/
|       ├── App.jsx                      # Main app — routing, step wizard, login
gate
|       ├── clients.js                   # Client profiles config file
|       ├── index.css                    # Global CSS variables and base styles
|       ├── index.js                     # React entry point
|       └── components/
|           ├── Login.jsx                # Login page (shared password)
|           ├── AdminPanel.jsx           # Admin panel — manage client profiles
|           ├── ClientSelect.jsx         # Step 1 — select client profile
|           ├── ProjectDetails.jsx       # Step 2 — project details form
|           ├── PersonnelDetails.jsx     # Step 3 — personnel details form
|           ├── EquipmentSelect.jsx      # Step 4 (manual) — select equipment
types
|           ├── PanelNumbers.jsx         # Step 5 (manual) — enter panel numbers
|           ├── IndexUpload.jsx          # Step 4 (index) — upload ITC index
spreadsheet
|           ├── ReviewGenerate.jsx       # Final step — review and generate ITCs
|           └── ui.js                   # Shared styled components (buttons,
fields etc.)
├── docker-compose.yml                   # Local development with Docker
├── .github/
|   └── workflows/
|       └── deploy.yml                   # CI/CD pipeline (GitHub Actions)
└── README.md                            # This file
```

How the App Works

User Flow

The app has two modes selectable at the start:

Mode 1 — Manual Entry

```
Login → Select Client → Project Details → Personnel → Equipment Selection → Panel
Numbers → Review & Generate
```

Mode 2 — Upload Index

```
Login → Select Client → Project Details → Personnel → Upload Index Spreadsheet → Review & Generate
```

Step-by-Step

1. **Login** — Enter the shared password (`CPPCommissioning@1`). Stored as an environment variable on Render (`ITC_PASSWORD`).
2. **Select Client** — Choose a saved client profile. This pre-fills all project details, site location, and personnel fields automatically. Profiles are managed by admins. Click "Skip" to fill in manually.
3. **Project Details** — CPP Project Name, Job No, Client Project Title, Client Project Number, Site Location, Date. Pre-filled if a client was selected.
4. **Personnel Details** — ITP Prepared By, Checked By (name, position, signature), Client Checked By. Pre-filled if a client was selected.

5a. **Equipment Selection (Manual mode)** — Choose equipment categories (Primary Equipment / Secondary & Protection) and tick the equipment types needed.

5b. **Panel Numbers (Manual mode)** — Enter panel numbers for each selected equipment type. One ITC is generated per panel number.

5c. **Upload Index (Index mode)** — Upload the ITC Index Excel spreadsheet. The app reads the **ITP SAT** tab and extracts:

- Column B — Functional location / equipment description (e.g. `275kV Circuit Breaker +C02-Q10`)
- Column I — Reference number (e.g. `TB3-EL-TC-30020`)
- Column Q — ITC Type (e.g. `Circuit Breaker`) — only rows with a value here are used
- Panel number is automatically extracted from Column B (everything starting with `+`)
- A preview table is shown with checkboxes to select/deselect individual ITCs

6. **Review & Generate** — Shows a summary of all details and ITCs to be generated. Click Generate to download.

Output Files

- **1 ITC** → downloads as a single `.xlsx` file named `TB3-EL-TC-30020 +C02-Q10.xlsx`
- **Multiple ITCs** → downloads as a `.zip` file named `[ProjectName]_ITCs.zip` with ITCs organised into subfolders by equipment type

What Gets Filled Into Each ITC

Cell	Value
A1	Title e.g. `Circuit Breaker +C02-Q10 SAT`
G9	CPP Project Name
G10	CPP Job No
G11	Functional Location (from Column B of index, or bay name in manual mode)
G12	Reference Number (from Column I of index)
G13	ITP Prepared By — Name
G14	ITP Prepared By — Position
G15	Date
G16	Checked By — Name
G17	Checked By — Position
G18	Checked By — Signature
G19	Checked By — Date
T9	Client Project Title
T10	Client Project Number
T11	Site Location
T13	Client Checked By — Name
T14	Client Checked By — Position

Page Setup

All generated ITCs are set to:

- **A4 Portrait**
- **Fit all columns to one page wide**
- Margins: 0.5" left/right, 0.75" top/bottom

Client Profiles

Client profiles are stored in `frontend/src/clients.js`. Each profile contains:

- Display name
- Template folder name (must match a folder in `backend/templates/`)
- All project details (CPP project name, job no, client project title etc.)
- All personnel details (prepared by, checked by, client checked by)
- Equipment lists (primary and secondary equipment available for this client)

Adding a New Client (Admin)

1. Click ⚙ **Admin** in the top right of the app
2. Enter the admin password (`CPPAdmin@1`)
3. Click **Add New Client Profile**
4. Fill in all details including the **Template Folder Name**
5. Create a matching folder in `backend/templates/[folder_name]/`
6. Add that client's ITC templates to the folder
7. Push to GitHub

Or edit `clients.js` directly and push to GitHub for a permanent change.

Client Template Folders

When generating ITCs, the app looks for templates in this order:

1. Client-specific folder: `backend/templates/[client_folder]/`
2. Primary ITCs fallback: `backend/templates/Primary_ITCs/`
3. Secondary ITCs fallback: `backend/templates/Protection_Secondary_ITCs/`
4. Templates root: `backend/templates/`

This means each client can have their own version of each template, and the app will automatically use the right one.

Equipment Types

Primary Equipment

Key	Label	Template File
`circuit_breaker`	Circuit Breaker	Circuit_Breaker.xlsx
`current_transformer`	Current Transformer	Current_Transformer.xlsx
`voltage_transformer`	Voltage Transformer	Voltage_Transformer.xlsx
`power_transformer`	Power Transformer	Power_Transformer.xlsx
`isolator`	Isolator	Isolator.xlsx
`surge_arrestor`	Surge Arrestor	Surge_Arrestor.xlsx
`neutral_ct`	Neutral CT	Neutral_CT.xlsx
`earth_switch`	Earth Switch	Earth_Switch.xlsx
`ows`	OWS	OWS.xlsx
`net`	NET	NET.xlsx
`aux_tf`	Auxiliary Transformer	Aux_TF.xlsx
`bess_pcs`	BESS PCS	BESS_PCS.xlsx
`ac_board`	AC Board	AC_Board.xlsx
`dc_panel`	DC Panel	DC_Panel.xlsx
`feeder_panel`	Feeder Panel	Feeder_Panel.xlsx

Secondary Equipment & Protection

Key	Label	Template File
`sel_751_feeder_relay`	SEL 751 Feeder Relay	SEL_751_Feeder_Relay.xlsx

Adding a New Equipment Template

1. **Prepare the template —** Save as `.xlsx` (not `.xlsm`). Ensure the standard header cells exist (G9–G19, T9–T14).

2. **Copy to templates folder**

```
backend/templates/Primary_ITCs/My_New_Equipment.xlsx
```

Or into a client-specific folder.

3. **Register in `backend/main.py`** — Add to `EQUIPMENT_TYPES` list and `TEMPLATE_FILENAMES` dict:

```python
EQUIPMENT_TYPES = [
    ...
    "my_new_equipment",
]
TEMPLATE_FILENAMES = {
    ...
    "my_new_equipment": "My_New_Equipment.xlsx",
}
```

4. **Register in `backend/generators/generic.py`** — Add to `LABELS` dict:

```python
LABELS = {
    ...
    "my_new_equipment": "My New Equipment",
}
```

5. **Add to frontend `frontend/src/components/EquipmentSelect.jsx`**:

```javascript
const PRIMARY = [
  ...
  { value: "my_new_equipment", label: "My New Equipment" },
];
```

6. **Update `frontend/src/clients.js`** — Add to each client's `primary_equipment` or `secondary_equipment` list.
7. **Push to GitHub**:

```powershell
cd C:\Users\cheron\Documents\itc-generator\itc-generator
git add .
git commit -m "Add My New Equipment template"
git push
```

Passwords

Password	Used For	Where to Change
`CPPCommissioning@1`	App login (all users)	Render → Environment → `ITC_PASSWORD`
`CPPAdmin@1`	Admin panel access	`frontend/src/App.jsx` — `ADMIN_PASSWORD` constant

> **Note:** The user login password is stored securely as an environment variable on Render. The admin password is currently hardcoded in the frontend — for production use this should be moved to the backend.

Security Notes

The current setup is suitable for internal use. It is **not** enterprise-grade:

- The backend API has no token-based authentication — only the frontend login is protected
- The admin password is hardcoded in the frontend JavaScript
- No audit trail of who generated what

Future improvements when rolling out company-wide:

- JWT token authentication on the backend
- Rate limiting
- IP allowlisting (CPP office IPs only)
- Proper admin authentication via backend
- Audit logging

Local Development

Prerequisites

- Python 3.11+
- Node.js 20+
- Git

Backend

```powershell
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Frontend

```powershell
cd frontend
npm install
# Set the backend URL
$env:REACT_APP_API_URL="http://localhost:8000"
npm start
```

Or with Docker

```powershell
docker-compose up --build
```

Deployment

Backend — Render

- **Service type:** Web Service
- **Root directory:** `backend`
- **Build command:** `pip install -r requirements.txt`

- **Start command:** `uvicorn main:app --host 0.0.0.0 --port $PORT`
- **Environment variable:** `ITC_PASSWORD = CPPCommissioning@1`

> ⚠️ Render free tier sleeps after 15 minutes of inactivity. First request after sleep takes ~30–60 seconds to wake up.

Frontend — Vercel

- **Root directory:** `frontend`
- **Environment variable:** `REACT_APP_API_URL = https://itc-generator.onrender.com`

Pushing Updates

```powershell
cd C:\Users\cheron\Documents\itc-generator\itc-generator
git add .
git commit -m "Description of change"
git push
```

Render and Vercel redeploy automatically on every push to `main`.

Index Spreadsheet Format

The app supports uploading an ITC Index spreadsheet to auto-populate ITCs. The spreadsheet must have a sheet named **ITP SAT** with:

Column	Content
B	Functional location / equipment description (e.g. `275kV Circuit Breaker +C02-Q10`)
I	Reference number (e.g. `TB3-EL-TC-30020`)
Q	ITC Type — must exactly match a supported equipment label (e.g. `Circuit Breaker`)

Only rows with a value in Column Q are processed. The panel number is automatically extracted from Column B (everything starting with `+`).

Supported ITC Types in Column Q

`Circuit Breaker`, `Current Transformer`, `Voltage Transformer`, `Power Transformer`, `Isolator`, `Surge Arrestor`, `Neutral CT`, `Earth Switch`, `OWS`, `NET`, `Auxiliary`

`Transformer`, `BESS PCS`, `AC Board`, `DC Panel`, `Feeder Panel`, `SEL 751 Feeder Relay`

Known Issues & Future Work

- [ ] Admin password should be moved to backend environment variable
- [ ] Backend needs JWT token auth for proper security
- [ ] Filter Bank template not yet added (no template available)
- [ ] Client profiles added via admin panel are session-only — not permanent (permanent clients must be added to `clients.js` and pushed)
- [ ] Render free tier sleep causes slow first load — upgrade to paid tier ($7/month) for always-on
- [ ] Custom domain not yet configured — using default Vercel URL
- [ ] No audit trail of ITC generation

Contact & Enquiries

For questions about the app contact the CPP commissioning team. For brand/marketing questions: Bianka Feo — bfeo@conpower.com.au

Consolidated Power Projects Australia Pty Ltd — A Quanta Services Company